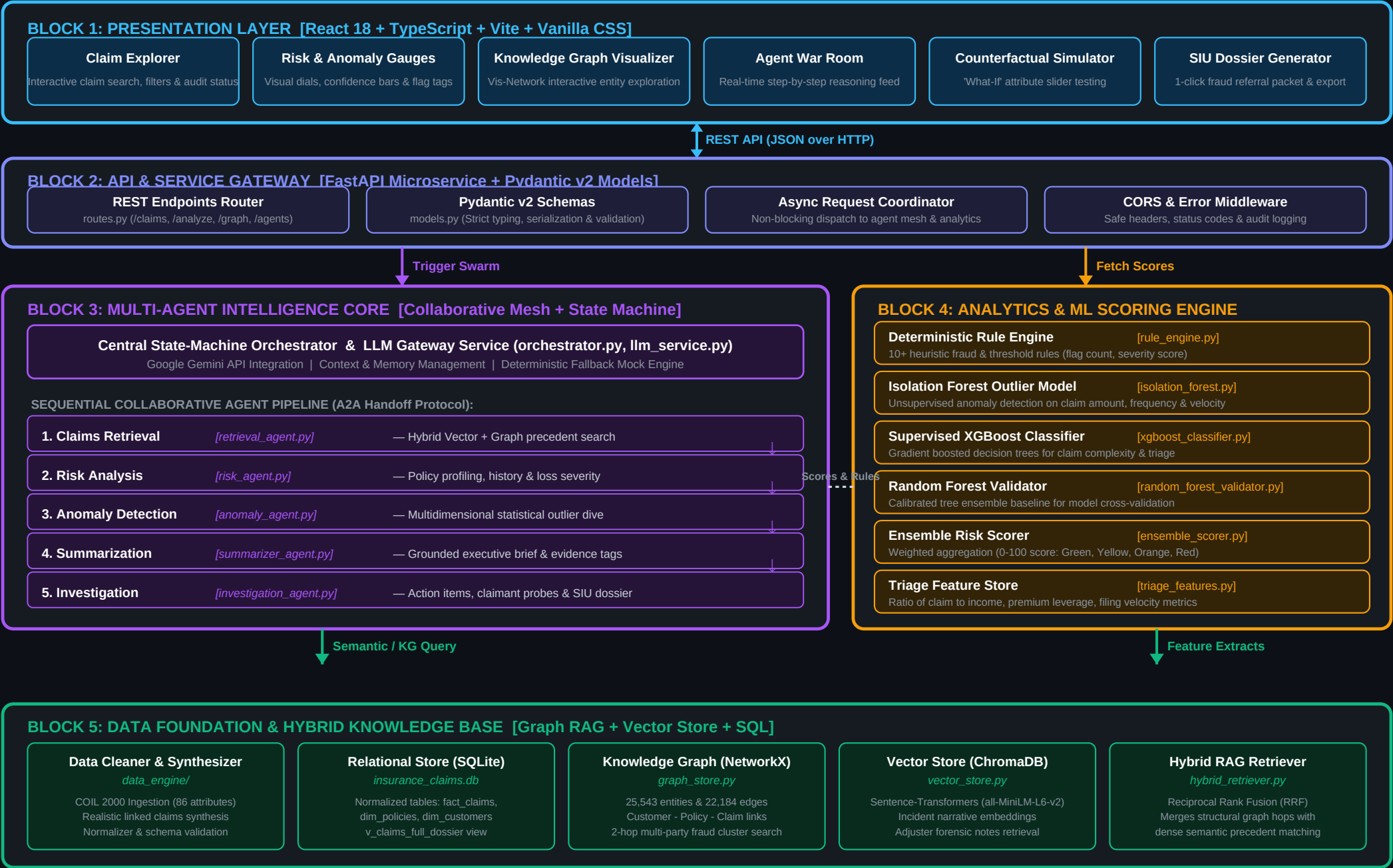


ANALYSTER — SYSTEM BLOCK ARCHITECTURE

AI-Powered Insurance Claims Intelligence Assistant | Modular Architectural Blueprint & Subsystem Blocks



ANALYSTER — BLOCK-BY-BLOCK FUNCTIONAL SPECIFICATION

Detailed architecture reference: Subsystem roles, input/output data contracts, and operational execution flow.

BLOCK 1: Presentation Layer (Reviewer Studio)

Tech Stack: React 18, TypeScript, Vite, Vanilla CSS, Vis-Network

CORE MISSION: Single-pane-of-glass workbench empowering claims adjusters and SIU investigators.

PRIMARY CODEBASE MODULES:

INPUT DATA: FastAPI REST API responses, SSE agent execution events, structured JSON claim dossiers.

frontend/src/App.tsx, views/ClaimExplorer.tsx, views/WarRoom.tsx, views/WhatIfSimulator.tsx

OUTPUT DATA: Adjuster decisions (Approve, Further Investigation, Escalate to SIU), audit annotations.

BLOCK 2: API & Gateway Layer

Tech Stack: FastAPI, Uvicorn, Pydantic v2, Python 3.10+

CORE MISSION: High-throughput asynchronous communication gateway ensuring strict contract validation.

PRIMARY CODEBASE MODULES:

INPUT DATA: HTTP GET / POST requests from frontend or external enterprise claim management systems.

backend/api/main.py, backend/api/routes.py, backend/api/models.py

OUTPUT DATA: Strictly typed Pydantic response payloads: ClaimDossier, RiskScorePayload, GraphNodesEdges.

BLOCK 3: Multi-Agent Intelligence Core

Tech Stack: Google Gemini 1.5 Pro / Flash, Custom State Machine, Python typing

CORE MISSION: Specialized collaborative swarm performing progressive reasoning via deterministic A2A handoffs.

PRIMARY CODEBASE MODULES:

INPUT DATA: Selected claim ID, policy record, precedent matches, ML anomaly scores, and rule signals.

orchestrator.py, retrieval_agent.py, risk_agent.py, anomaly_agent.py, summarizer_agent.py, investigation_agent.py

OUTPUT DATA: Synthesized executive brief, evidence citations, interview probes, and formal SIU referral dossier.

BLOCK 4: Analytics & Machine Learning Engine

Tech Stack: Scikit-Learn (Isolation Forest), XGBoost, NumPy, Pandas

CORE MISSION: Dual-method scoring: heuristic business rule evaluation and unsupervised anomaly detection.

PRIMARY CODEBASE MODULES:

INPUT DATA: Engineered tabular features: claim-to-income ratio, filing velocity, premium volume, claim frequency.

rule_engine.py, isolation_forest.py, ensemble_scorer.py, triage_features.py

OUTPUT DATA: Normalized Anomaly Score [-1 to +1], Composite Risk Score [0 to 100], 4-tier triage priority.

BLOCK 5: Data Foundation & Hybrid Knowledge Base

Tech Stack: ChromaDB (Dense Vectors), NetworkX (Property Graph), SQLite (Relational Store)

CORE MISSION: Unified enterprise data layer combining structured policies with semantic narrative indexing.

PRIMARY CODEBASE MODULES:

INPUT DATA: COIL 2000 benchmark dataset, synthesized incident narratives, policy portfolio tables.

claims_synthesizer.py, vector_store.py, graph_store.py, hybrid_retriever.py

OUTPUT DATA: Dense 384-d narrative embeddings, 2-hop graph neighborhood subgraphs, normalized SQL views.

END-TO-END EXECUTION FLOW SUMMARY (7 STAGES)

1. Ingestion

COIL 2000 + Synthesizer

2. Storage

SQLite + Chroma + Graph

3. Scoring

Isolation Forest + Rules

4. Orchestration

State-Machine Swarm

5. Hybrid RAG

Vector + 2-Hop Graph

6. Synthesis

Grounded Dossier + SIU

7. Reviewer

Human-in-the-Loop